

Proyecto Grupal I: Arquitectura del Set de Instrucciones (ISA) Específica tipo RISC para Aplicaciones de Seguridad Informática

Brayan Esteban Alpízar Elizondo
Mariana González Sanabria
Frederick Obando Solano
Diego Salas Ovares

Arquitectura de Computadores I
Instituto Tecnológico de Costa Rica

1 de mayo de 2026

1. Descripción General

F32IS es una arquitectura RISC de 32 bits orientada a aplicaciones de criptografía y seguridad. Sigue el modelo de ejecución registro-registro con direccionamiento por desplazamiento en formato little-endian.

Propiedad	Valor
Tipo	RISC (Reduced Instruction Set Computer)
Clase	Registro-Registro
Direccionamiento	Little-endian, por desplazamiento
Tamaño de palabra	32 bits
Tipos de datos	Enteros
Extensiones	I (Enteros), S (Seguridad)
Especificaciones	Criptografía, Seguridad

2. Banco de registros

2.1. Banco de Registros General

La arquitectura cuenta con 32 registros de propósito general definidos con una dirección de 5 bits. Los registros marcados como solo lectura no pueden ser escritos por instrucciones de usuario.

Dirección	Nombre	Descripción
0	zero	Constante 0 (<i>read-only</i>)
1	ra	Return address (dirección de retorno)
2	sp	Stack pointer
3	pc	Program counter
4	lr	Link register (<i>read-only</i>)
5	p0	Argumento y retorno de función
6–13	p1–p8	Argumentos de función
14–29	r0–r15	Registros de propósito general
30	delta	Constante 0x9E3779B9 (<i>read-only</i>)
31	max	Constante 0xFFFFFFFF (<i>read-only</i>)

2.2. Banco de Registros Seguro

8 registros exclusivos para la extensión de seguridad, con dirección de 3 bits. Solo accesibles tras autenticación.

Dirección	Nombre	Descripción
0	ax	Constante 0 (<i>read-only</i>)
1	bx	Propósito general
2	cx	Propósito general
3	dx	Propósito general
4	ex	Propósito general
5	fx	Propósito general
6	gx	Propósito general
7	hx	Propósito general

3. Formato de Instrucciones

Todas las instrucciones tienen una longitud de 32 bits. En todas ellas, el bit 0 corresponde al bit P (seguridad), mientras que los bits 1–5 representan el código de operación (opcode).

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	func7							rm					rn				rd				func4				opcode				P			
I	imm12											rn				rd				func4				opcode				P				
M	imm12											rn				rd				W	H	B	S	opcode				P				
B	imm12[11:5]							rm					rn				imm12[4:0]				cond				opcode				P			
J/F	imm21[20:4]															rd				imm21[3:0]				opcode				P				
S	imm20																			0	0	func4				opcode				P		
V	imm16															sn			sd			W	H	B	S	opcode				P		
PR	func7							func3			sf		sm		sn			sd			func4				opcode				P			
PI	imm16															sn			sd			func4				opcode				P		
T	0	0	0	0	0	0	0	0	0	0	0	0	rn				0	0	sd			func4				opcode				P		
	0	0	0	0	0	0	0	0	0	0	0	0	sm			0	rd															

3.1. Tipo R — Aritmética/Lógica Registro-Registro

Sintaxis: op rd, rn, rm — Excepción: mov rd, rn

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
func4	6–9	Funcionalidad
rd	10–14	Registro destino
rn	15–19	Registro fuente 1
rm	20–24	Registro fuente 2
func7	25–31	Funcionalidad extra

3.2. Tipo I — Aritmética/Lógica Registro-Inmediato

Sintaxis: op rd, rn, imm — Excepción: movi rd, imm

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
func4	6–9	Funcionalidad
rd	10–14	Registro destino
rn	15–19	Registro fuente
imm12	20–31	Inmediato de 12 bits

3.3. Tipo M — Operaciones de Memoria

Sintaxis: op rd, $\pm$ imm(rn)

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
S	6	Selección de Operación (0: suma, 1:resta)
B	7	Movimiento de byte
H	8	Movimiento de media palabra
W	9	Movimiento de palabra
rd	10–14	Registro dato
rn	15–19	Registro base
imm12	20–31	Desplazamiento

3.4. Tipo B — Control de Flujo

Sintaxis: op rn, rm, imm

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
cond	6–9	Condición de salto
imm12[4:0]	10–14	inmediato de 12 bits (bits bajos)
rn	15–19	Registro fuente 1
rm	20–24	Registro fuente 2
imm12[11:5]	25–31	inmediato de 21 bits (bits altos)

3.5. Tipo J/F — Salto Largo relativo al PC

Sintaxis: jal rd, imm

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
imm21[3:0]	6–9	inmediato de 21 bits (bits bajos)
rd	10–14	Dirección de registro
imm21[20:4]	15–31	inmediato de 21 bits (bits altos)

3.6. Tipo S — Sesión de Seguridad

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
func4	6–9	Funcionalidad
const	10–11	Constante 0
imm20	12–31	inmediato de 20 bits (Contraseña)

3.7. Tipo V — Bóveda Segura

Sintaxis: op sd, $\pm$ imm(sn)

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
S	6	Selección de Operación (0: suma, 1:resta)
B	7	Movimiento de byte
H	8	Movimiento de media palabra
W	9	Movimiento de palabra
sd	10–12	Registro destino seguro
sn	13–15	Registro base seguro
imm16	16–31	Desplazamiento

3.8. Tipo PR — Aritmética/Lógica sobre Registros Seguros

Sintaxis: op sd, sn, sm — op sd, sn, sm, sf

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
func4	6–9	Funcionalidad primaria
sd	10–12	Registro destino
sn	13–15	Registro fuente 1
sm	16–18	Registro fuente 2
sf	19–21	Registro fuente 3
func3	22–24	Funcionalidad secundaria
func7	25–31	Funcionalidad extra

3.9. Tipo PI — Aritmética/Lógica sobre Registros Seguros e Inmediatos

Sintaxis: op sd, sn, imm

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
func4	6–9	ALU
sd	10–12	Registro destino
sn	13–15	Registro fuente
imm16	16–31	Inmediato de 16 bits

3.10. Tipo T — Transferencia

Instrucciones: send sd, rn — recv rd, sm

Campo	Bits	Descripción
P	0	Bit de seguridad
opcode	1–5	Código de instrucción
func4	6–9	Funcionalidad
sd	10–12	Registro Seguro Destino
rd	10–14	Registro Destino
rn	15–19	Registro Fuente
sm	16–18	Registro Seguro Fuente

4. Set de Instrucciones

A continuación, se muestran todas las instrucciones del set según su formato. Las notaciones R[...], M[...], S[...] y V[...] significan accesos a registros, memoria, registros seguros y bóveda,

respectivamente.

4.1. Extensión I — Enteros

4.1.1. Tipo R

Instrucción	Operación	Descripción
add rd, rn, rm	$R[rd] \leftarrow R[rn] + R[rm]$	Suma
sub rd, rn, rm	$R[rd] \leftarrow R[rn] - R[rm]$	Resta
mul rd, rn, rm	$R[rd] \leftarrow R[rn] \times R[rm]$	Multiplicación
div rd, rn, rm	$R[rd] \leftarrow R[rn] \div R[rm]$	División
mod rd, rn, rm	$R[rd] \leftarrow R[rn] \% R[rm]$	Módulo
and rd, rn, rm	$R[rd] \leftarrow R[rn] \& R[rm]$	AND bit a bit
orr rd, rn, rm	$R[rd] \leftarrow R[rn] R[rm]$	OR bit a bit
xor rd, rn, rm	$R[rd] \leftarrow R[rn] \wedge R[rm]$	XOR bit a bit
sll rd, rn, rm	$R[rd] \leftarrow R[rn] \ll R[rm]$	Shift left lógico
srl rd, rn, rm	$R[rd] \leftarrow R[rn] \gg R[rm]$	Shift right lógico
seq rd, rn, rm	$R[rd] \leftarrow (R[rn] == R[rm])$	Comparación igual
mov rd, rn	$rd \leftarrow R[rn]$	Copia de registro

4.1.2. Tipo I

Instrucción	Operación	Descripción
addi rd, rn, imm	$R[rd] \leftarrow R[rn] + imm$	Suma con inmediato
subi rd, rn, imm	$R[rd] \leftarrow R[rn] - imm$	Resta con inmediato
muli rd, rn, imm	$R[rd] \leftarrow R[rn] \times imm$	Multiplicación
divi rd, rn, imm	$R[rd] \leftarrow R[rn] \div imm$	División
modi rd, rn, imm	$R[rd] \leftarrow R[rn] \% imm$	Módulo
andi rd, rn, imm	$R[rd] \leftarrow R[rn] \& imm$	AND bit a bit
orri rd, rn, imm	$R[rd] \leftarrow R[rn] imm$	OR bit a bit
xori rd, rn, imm	$R[rd] \leftarrow R[rn] \wedge imm$	XOR bit a bit
slli rd, rn, imm	$R[rd] \leftarrow R[rn] \ll imm$	Shift left lógico
srli rd, rn, imm	$R[rd] \leftarrow R[rn] \gg imm$	Shift right lógico
seqi rd, rn, imm	$R[rd] \leftarrow (R[rn])$	Comparación
movi rd, imm	$R[rd] \leftarrow imm$	Carga inmediato

4.1.3. Tipo M — Memoria

Instrucción	Operación	Descripción
ldw rd, $\pm$ imm(rn)	$R[rd] \leftarrow M[R[rn] \pm imm]_{31:0}$	Carga palabra
ldh rd, $\pm$ imm(rn)	$R[rd] \leftarrow M[R[rn] \pm imm]_{15:0}$	Carga media palabra
ldb rd, $\pm$ imm(rn)	$R[rd] \leftarrow M[R[rn] \pm imm]_{7:0}$	Carga byte
stw rd, $\pm$ imm(rn)	$M[R[rn] \pm imm] \leftarrow R[rd]_{31:0}$	Almacena palabra
sth rd, $\pm$ imm(rn)	$M[R[rn] \pm imm] \leftarrow R[rd]_{15:0}$	Almacena media palabra
stb rd, $\pm$ imm(rn)	$M[R[rn] \pm imm] \leftarrow R[rd]_{7:0}$	Almacena byte

4.1.4. Tipo B y J

En estos tipos de instrucción el valor del inmediato representa la distancia de salto hacia una instrucción con una etiqueta (generalmente calculada por el compilador). Su cálculo se basa en la siguiente operación: $\text{instr}[j] - \text{instr}[i]$; donde i representa el índice de la siguiente instrucción a la actual ($PC + 4$ y j) representa el índice de la instrucción a la que se desea saltar.

Instrucción	Condición	Descripción
beq rn, rm, imm	$R[rn] == R[rm]$	Salta si igual
bne rn, rm, imm	$R[rn] \neq R[rm]$	Salta si distinto
bgt rn, rm, imm	$R[rn] > R[rm]$	Mayor
blt rn, rm, imm	$R[rn] < R[rm]$	Menor
bge rn, rm, imm	$R[rn] \geq R[rm]$	Mayor o igual
ble rn, rm, imm	$R[rn] \leq R[rm]$	Menor o igual
jal rd, imm	$R[rd] \leftarrow PC; PC \leftarrow PC + imm$	Salto y enlace

4.1.5. Tipo S

Instrucción	Operación	Descripción
login imm	$lr \leftarrow (imm == password)$	Inicio de sesión
quit	$lr \leftarrow 0$	Cierre de sesión

4.2. Extensión S — Seguridad

4.2.1. Tipo PR

Instrucción	Operación	Descripción
padd sd, sn, sm	$S[sd] \leftarrow S[sn] + S[sm]$	Suma
psub sd, sn, sm	$S[sd] \leftarrow S[sn] - S[sm]$	Resta
pmul sd, sn, sm	$S[sd] \leftarrow S[sn] \times S[sm]$	Multiplicación
pdiv sd, sn, sm	$S[sd] \leftarrow S[sn] \div S[sm]$	División
pmod sd, sn, sm	$S[sd] \leftarrow S[sn] \% S[sm]$	Módulo
pand sd, sn, sm	$S[sd] \leftarrow S[sn] \& S[sm]$	AND
porr sd, sn, sm	$S[sd] \leftarrow S[sn] S[sm]$	OR
pxor sd, sn, sm	$S[sd] \leftarrow S[sn] \wedge S[sm]$	XOR
pseq sd, sn, sm	$S[sd] \leftarrow (S[sn] == S[sm])$	Comparación
pmov sd, sn	$S[sd] \leftarrow S[sn]$	Copia

4.2.2. Tipo PI

Instrucción	Operación	Descripción
paddi sd, sn, imm	$S[sd] \leftarrow S[sn] + imm$	Suma
psubi sd, sn, imm	$S[sd] \leftarrow S[sn] - imm$	Resta
pmuli sd, sn, imm	$S[sd] \leftarrow S[sn] \times imm$	Multiplicación
pdivi sd, sn, imm	$S[sd] \leftarrow S[sn] \div imm$	División
pmodi sd, sn, imm	$S[sd] \leftarrow S[sn] \% imm$	Módulo
pandi sd, sn, imm	$S[sd] \leftarrow S[sn] \& imm$	AND
porri sd, sn, imm	$S[sd] \leftarrow S[sn] imm$	OR
pxori sd, sn, imm	$S[sd] \leftarrow S[sn] \wedge imm$	XOR
pseqi sd, sn, imm	$S[sd] \leftarrow (S[sn] == imm)$	Comparación
pmovi sd, imm	$S[sd] \leftarrow imm$	Carga

4.2.3. Tipo V

Instrucción	Operación	Descripción
ldvw sd, $\pm imm(sn)$	$S[sd] \leftarrow V[S[sn] \pm imm]_{31:0}$	Carga palabra
ldvh sd, $\pm imm(sn)$	$S[sd] \leftarrow V[S[sn] \pm imm]_{15:0}$	Carga media palabra
ldvb sd, $\pm imm(sn)$	$S[sd] \leftarrow V[S[sn] \pm imm]_{7:0}$	Carga byte
stvw sd, $\pm imm(sn)$	$V[S[sn] \pm imm] \leftarrow S[sd]_{31:0}$	Almacena palabra
stvh sd, $\pm imm(sn)$	$V[S[sn] \pm imm] \leftarrow S[sd]_{15:0}$	Almacena media palabra
stvb sd, $\pm imm(sn)$	$V[S[sn] \pm imm] \leftarrow S[sd]_{7:0}$	Almacena byte

4.2.4. Tipo T

Instrucción	Operación	Descripción
send sd, rn	$S[sd] \leftarrow R[rn]$	Copia a registro seguro contenido de registro general
recv rd, sm	$R[rd] \leftarrow S[sm]$	Copia a registro general contenido de registro seguro

4.3. Pseudo Instrucciones

Tipo	Instrucción	Descripción	Representación
F	call imm	$R[ra] \leftarrow PC; PC \leftarrow PC + imm$	jal ra, imm
B	beqz rn, imm	if ($R[rn] == 0$) $PC \leftarrow PC + (imm \ll 2)$	beq rn, zero, imm
J	jmp imm	$R[zero] \leftarrow PC; PC \leftarrow PC + imm$	jal zero, imm
R	ret	$R[pc] \leftarrow R[ra]$	mov pc, ra
R	nop	$R[zero] \leftarrow R[zero] + R[zero]$	add zero, zero, zero
R	seqz rd, rn	$R[rd] \leftarrow (R[rn] == R[zero])$	seq rd, rn, zero
I	la rd, addr	$R[rd] \leftarrow addr$	movi rd, addr
I	li rd, imm	$R[rd] \leftarrow imm$	movi rd, imm
PI	pla sd, addr	$S[sd] \leftarrow addr$	pmovi sd, addr
PI	pli sd, imm	$S[sd] \leftarrow imm$	pmovi sd, imm

5. Encodificación

5.1. Especificación de Tipos Instrucciones

Opcode	Instrucciones	Tipo
00000	add, sub, etc	R
00001	addi, subi, etc	I
00010	padd, psub, etc	PR
00011	paddi, psubi, etc	PI
00100	ldw, ldh, ldb	M
00101	stw, sth, stb	M
00110	ldvw, ldvh, ldvb	V
00111	stvw, stvh, stvb	V
01000	beq, bge, etc	B
01001	jal, call	J/F
10000	send, recv	T
10001	login, quit	S

5.2. Especificación de Operación para Instrucciones T

func4	Operación
0000	send
0001	recv

5.3. Especificación de Operación para Instrucciones S

func4	Operación
0000	login
0001	quit

5.4. Especificación de Operación para ALU Primaria

func4	Operación
0000	SLL ($\ll$)
0001	SRL ($\gg$)
0010	ADD (+)
0011	SUB ($-$)
0100	MUL ($\times$)
0101	DIV ($\div$)
0110	MOD (%)
0111	AND (&)
1000	OR ()
1001	XOR ($\oplus$)
1010	SEQ

5.5. Especificación de Operación para ALU Secundaria

Esta encodificación es exclusiva para las instrucciones tipo PR que utilizan dos operaciones.

func3	Operación
000	ADD (+)
001	XOR ($\oplus$)

5.6. Especificación de Condición de Salto para Instrucciones B

cond	Comparación
0001	BEQ (==)
0010	BNE (≠)
0011	BGT (>)
0100	BLT (<)
0101	BGE (≥)
0110	BLE (≤)

6. Convenciones

6.1. Convención para Definición de Funciones

Las funciones se invocan con `call` y deben respetar la convención de registros para garantizar integridad entre llamadas.

Registros	Uso	Responsabilidad
p0	Retorno y primer argumento	Caller
p1-p8	Argumentos adicionales	Caller
r0-r15	Propósito general	Callee (guardar/restaurar)
ra	Dirección de retorno	Callee (guardar si realiza llamadas)
sp	Stack pointer	Callee (debe restaurarlo antes de retornar)

Ejemplo:

```
__init__:
    movi r1, 4
    movi p0, 7
    add p1, p0, r1
    call sum
    mul r1, p0, r1

// sum(a,b) => p0:a, p1:b, p0:a*2+b
sum:
    addi sp, sp, 8
    stw ra, 0(sp)
    stw r1, 4(sp)

    li r1, 2
    mul r1, p0, r1
    add p0, r1, p1
```

```
ldw r1, 4(sp)
ldw ra, 0(sp)
addi sp, sp, -8
ret
```

6.2. Convención de Instrucciones en Modo Administrador

Para acceder al hardware de seguridad es necesario iniciar sesión mediante la instrucción `login`, la cual recibe una contraseña de 5 dígitos hexadecimales. Si la contraseña es correcta, el hardware de seguridad se habilita.

Las instrucciones de tipo **PR**, **PI**, **V** y **T** requieren que exista una sesión activa para poder ejecutarse. De igual forma, aquellas instrucciones que inicien con el símbolo `@` únicamente se ejecutan si el hardware de seguridad se encuentra habilitado.

Una vez finalizado el uso del hardware de seguridad, la sesión debe cerrarse utilizando la instrucción `quit`. El estado de la sesión se almacena en el registro `lr`, el cual indica si el acceso seguro está habilitado o no.

En caso de no cerrar la sesión de forma explícita, el hardware de seguridad implementa un límite de ciclos, tras el cual se deshabilita automáticamente.

A continuación se presenta un ejemplo de uso:

```
addi r1, r0, 4
login 0xBEEF0    # Inicio de sesion
li r2, 100
@mul r3, r1, r2
send ax, r0
send bx, r3
paddi cx, ax, 1
@psub dx, bx, cx
quit            # Cierre de sesion
recv dx, r4     # Esta instruccion no se ejecuta
```